

# Operational Depth: A Unified Iterative-Reductive Framework for Fundamental Arithmetic

Pearl Bipin Pulickal

*Department of Electronics Engineering, NIT Goa*

January 8, 2026

## Abstract

Standard arithmetic operations are typically defined axiomatically (e.g., Peano arithmetic) or via set-theoretic construction. While rigorous, these static definitions often obscure the algorithmic continuity between addition, multiplication, and exponentiation—a continuity that is fundamental to computer architecture. This paper proposes an alternative framework: viewing fundamental operations not as static truths, but as iterative processes characterized by their “Operational Depth.” We formalize a duality between *Generative* (constructive) and *Reductive* (inverse) processes relative to an algebraic identity element. By defining division not merely as inverse multiplication, but as the iteration count required to reduce a value to the additive identity, we unify the arithmetic hierarchy under a procedural lens. Furthermore, we extend this framework to continuous domains via differential equations and demonstrate that square roots can be interpreted as symmetric fixed points of this reductive process. This perspective bridges abstract algebra with the procedural intuition of algorithmic engineering.

## 1 Introduction

In the philosophy of mathematics, arithmetic is often treated as a set of static truths:  $4 \times 3 = 12$  is a fact derived from the properties of rings and fields. However, in computer architecture and engineering, arithmetic is a *process*. An Arithmetic Logic Unit (ALU) does not simply recall facts from a lookup table; it executes algorithms. It computes a product through iterative shifting and adding, and a quotient through iterative subtraction or restoration.

There exists a disconnect between the pedagogical treatment of arithmetic (where operations are distinct, axiomatically defined categories) and the operational reality (where higher-order operations are iterations of lower-order ones). Ideally, a unified theory should explain why exponentiation relates to multiplication in the same way multiplication relates to addition, without requiring distinct axioms for each level.

This paper formalizes such a perspective, proposing a framework based on **Operational Depth**. The core thesis suggests that the hierarchy of hyper-operations can be operationally sketched by two complementary trajectories relative to an Origin (Identity Element):

1. **Construction (Generative Depth):** The iterations of a generator required to transition from the Origin to a target. (*“How many times must I add  $x$  to 0 to create  $y$ ?”*)

2. **Reduction (Termination Depth):** The iterations of an inverse operator required to return a target value to the Origin. (“How many times must I subtract  $x$  from  $y$  to return to 0?”)

### 1.1 Distinction from Prior Frameworks

It is important to distinguish this work from the recursive definitions found in Goodstein’s *Recursive Number Theory* [3] or the Ackermann function. While those frameworks build higher operations (hyperoperations) *upwards* through recursion to prove logical consistency, this framework analyzes operations *downwards* through the lens of computational cost. We do not seek to redefine the axioms of number theory, but rather to model the “energy” (iterations) required to generate or destroy a value relative to an identity element.

## 2 The Formal Framework

We define an *Iterative Arithmetic Context* to formalize the notion of construction and destruction.

**Definition 2.1** (Arithmetic Context). A context is a triple  $\mathcal{A} = (S, \oplus, \mathcal{O})$ , where:

- $S$  is a totally ordered set (e.g.,  $\mathbb{R}^+, \mathbb{Z}$ ).
- $\oplus : S \times S \rightarrow S$  is a binary generator operation (associative and commutative).
- $\mathcal{O} \in S$  is the identity element for  $\oplus$ .

Assuming  $\oplus$  is invertible, we define the unique inverse operation  $\ominus$  (the reductive operator) such that  $(a \oplus b) \ominus b = a$ .

### 2.1 Operational Depth

We propose that the magnitude of an operation is defined by the count of iterations in a cycle starting from and returning to the identity.

#### Definition: Operational Depth

**Operational Depth** is defined as the scalar magnitude  $\mathcal{T}$  representing the quantification of iterations required for a generator operation  $\oplus$  to transition an operand  $b$  from an identity element  $\mathcal{O}$  to a target value  $a$  (Generative), or conversely, for an inverse operator  $\ominus$  to reduce  $a$  by  $b$  back to  $\mathcal{O}$  (Termination).

### 2.2 Discrete Formulation

**Definition 2.2** (Constructive Depth). Let  $b \in S$  and  $n \in \mathbb{N}$ . The result of the operation of rank  $R + 1$  is  $c_n$ , where  $c_0 = \mathcal{O}$  and  $c_{k+1} = c_k \oplus b$ .

$$\Phi(b, n) = c_n = \underbrace{b \oplus b \oplus \dots \oplus b}_{n \text{ times}}$$

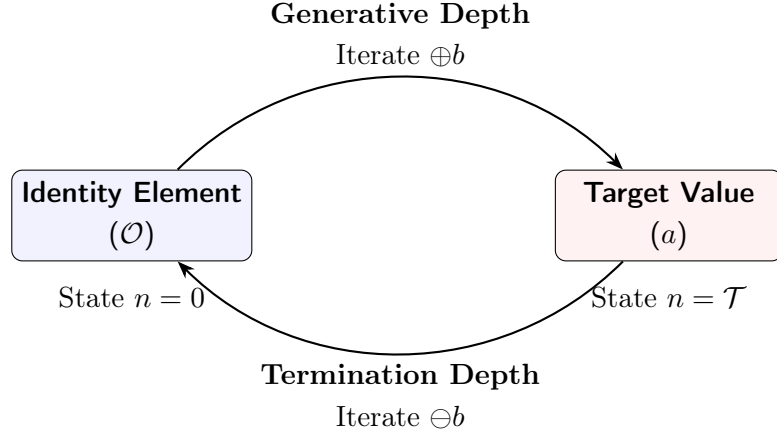


Figure 1: The Cycle of Operational Depth. Generative depth builds from the identity; Termination depth reduces to the identity.

**Definition 2.3** (Termination Depth). Let  $a, b \in S$ . The destructive sequence  $\{d_k\}$  is defined by  $d_0 = a$  and  $d_{k+1} = d_k \ominus b$ . The Termination Depth, denoted  $\mathcal{T}(a, b)$ , is the smallest integer  $n \geq 0$  such that the sequence reaches the Origin:

$$d_n = \mathcal{O}$$

*Example 2.1* (Fully Worked Discrete Example). Consider the Additive Context  $\mathcal{A} = (\mathbb{Z}, +, 0)$ . We wish to find the Termination Depth  $\mathcal{T}(12, 3)$ .

- Generator: Addition (+). Identity: 0. Inverse: Subtraction (−).
- Start:  $d_0 = 12$ .
- Iterations:  $12 \xrightarrow{-3} 9 \xrightarrow{-3} 6 \xrightarrow{-3} 3 \xrightarrow{-3} 0$ .

Since  $d_4 = \mathcal{O}$ , the Operational Depth is 4. In standard arithmetic, this corresponds to  $12 \div 3 = 4$ .

### 3 The Hierarchy of Operations

The power of this framework appears when we change the Origin. By shifting the perspective from “Nothing is Zero” to “Nothing is One,” we naturally derive exponentiation and logarithms without defining new axioms.

#### 3.1 Level 1: The Additive Context

**Context:**  $\mathcal{A}_1 = (\mathbb{R}, +, 0)$ .

- **Multiplication (Construction):**

$$c_n = 0 + \underbrace{b + \cdots + b}_n = n \times b$$

- **Division (Termination):** We seek  $n$  such that  $d_n = a - n \cdot b = 0$ .

**Proposition 3.1.** In  $(\mathbb{R}, +, 0)$ , the Termination Depth  $\mathcal{T}(a, b)$  is isomorphic to division  $a/b$ , provided  $b \neq 0$ .

### 3.2 Level 2: The Multiplicative Context

To ascend the hierarchy, the generator becomes Multiplication. **Context:**  $\mathcal{A}_2 = (\mathbb{R}^+, \times, 1)$ .

*Remark 3.1* (Singularity at Zero). Unlike the additive context, the multiplicative context implies a singularity at 0. Zero has no multiplicative inverse and cannot be “destroyed” to reach 1. Thus, we restrict the domain  $S$  to  $\mathbb{R}^+$  to preserve group structure.

- **Exponentiation (Construction):** Starting from the identity (1), we apply the generator (multiply by  $b$ )  $n$  times.

$$c_n = 1 \times \underbrace{b \times \cdots \times b}_n = b^n$$

- **Logarithms (Termination):** The reductive operator is division. We reduce  $a$  by dividing by  $b$  until we reach the identity (1).

$$d_0 = a, \quad d_{k+1} = d_k / b$$

We seek termination at the Origin ( $d_n = 1$ ):

$$\frac{a}{b^n} = 1 \implies a = b^n \implies n = \log_b a$$

**Corollary 3.2.** *The logarithm  $\log_b a$  is formally defined as the Termination Depth of  $a$  relative to base  $b$  in the multiplicative context  $\mathcal{A}_2$ .*

### 3.3 Summary of the Hierarchy

| Level | Context        | Origin | Generative (Construction)                                        | Reductive (Termination)                                    |
|-------|----------------|--------|------------------------------------------------------------------|------------------------------------------------------------|
| 1     | Additive       | 0      | <b>Multiplication</b> ( $n \times b$ )<br>(Add $b$ , $n$ times)  | <b>Division</b> ( $a/b$ )<br>(Subtract $b$ until 0)        |
| 2     | Multiplicative | 1      | <b>Exponentiation</b> ( $b^n$ )<br>(Multiply by $b$ , $n$ times) | <b>Logarithm</b> ( $\log_b a$ )<br>(Divide by $b$ until 1) |

Table 1: The Operational Depth Hierarchy showing the unification of operations.

## 4 Symmetric Termination: The Square Root

Standard definitions often treat roots as fractional exponents ( $a^{1/2}$ ). However, our framework allows for a more algorithmic definition based on *operational symmetry*. Typically, an operation involves an operand ( $b$ ) and a depth count ( $n$ ). What if the operand and the count are the same?

**Definition 4.1** (Symmetric Termination). Let  $\mathcal{A}$  be an arithmetic context. A value  $x$  is the **Symmetric Root** of  $a$  if the Termination Depth of  $a$  by  $x$  is exactly  $x$ . That is:

$$\mathcal{T}(a, x) = x$$

Let us apply this to the fundamental Additive Context  $\mathcal{A}_1$ . We are looking for an  $x$  such that if we subtract  $x$  from  $a$ , it takes exactly  $x$  steps to reach 0.

From Proposition 3.1, we know that in this context,  $\mathcal{T}(a, x) = a/x$ . Imposing the symmetry condition:

$$\frac{a}{x} = x \implies a = x^2 \implies x = \sqrt{a}$$

*Remark 4.1* (The Fixed Point Interpretation). This provides a coherent verbal definition of the square root entirely within the additive domain: "*The square root of  $a$  is the number  $x$  which must be subtracted from  $a$  exactly  $x$  times to yield zero.*" This connects roots to additive processes without explicitly invoking Level 2 geometry (squares) or fractional exponents.

## 5 Continuous Extension: From Discrete to Differential

As noted in Example 2.1, discrete depth fails for non-integers (e.g., destroying 13 by 3 skips the origin). To resolve this, we model the “steps” as a continuous flow via differential equations. The iteration index  $n$  becomes a continuous time variable  $t$ .

### 5.1 Additive Continuity

- **Discrete Difference:**  $d_{k+1} - d_k = -b$ .
- **Continuous Flow:** Let  $y(t)$  represent value at “time”  $t$ . The destructive process is constant rate decay:

$$\frac{dy}{dt} = -b, \quad y(0) = a$$

Solving for  $y(t) = 0$  yields  $t = a/b$ .

### 5.2 Multiplicative Continuity

For the multiplicative context, the discrete step is division:  $d_{k+1} = d_k/b$ . In a continuous domain, division by  $b$  corresponds to subtracting a logarithm. The rate of change is proportional to the current value (exponential decay).

*Derivation of Continuous Logarithms.* Consider the limit of discrete scaling. The reduction step is  $y(t + \Delta t) = y(t) \cdot b^{-\Delta t}$ . The difference quotient is:

$$\frac{y(t + \Delta t) - y(t)}{\Delta t} \approx y(t) \frac{b^{-\Delta t} - 1}{\Delta t}$$

Taking the limit  $\Delta t \rightarrow 0$  and using the identity  $\lim_{h \rightarrow 0} \frac{b^h - 1}{h} = \ln b$ :

$$\frac{dy}{dt} = -y(t) \ln b$$

Solving this differential equation with initial condition  $y(0) = a$ :

$$\int \frac{1}{y} dy = \int -\ln b dt \implies \ln y = -t \ln b + C$$

$$y(t) = ae^{-t \ln b} = ab^{-t}$$

We seek the time  $t$  where the value reaches the Multiplicative Origin (1):

$$1 = ab^{-t} \implies b^t = a \implies t = \log_b a$$

□

This validates that the continuous formulation of Operational Depth aligns perfectly with standard logarithms, providing a calculus-based justification for the hierarchy.

## 6 Computational Implications

This framework is not merely a pedagogical curiosity; it mirrors the physical reality of computation. In digital electronics, specifically within an Arithmetic Logic Unit (ALU), operations are rarely performed via lookup tables for large numbers.

- **Hardware Division:** Often implemented as “restoring division,” which is physically a process of iterative subtraction and shifting—tracking the termination depth at binary scales.
- **CORDIC Algorithms:** Many hardware implementations for logarithms and trigonometric functions rely on iterative rotation and reduction methods [4], which align closely with the “Termination Depth” concept rather than Taylor series expansions.

By framing arithmetic as “processes of destruction to an origin,” we bridge the gap between abstract algebra (where operations are static mappings) and computer engineering (where operations are temporal cycles of state change).

## 7 Conclusion

This paper has presented **Operational Depth** as a unified conceptual framework for arithmetic. By shifting focus from static axioms to dynamic iterative processes, we observe that:

1. **Multiplication and Division** are paired constructive/reductive processes relative to the Additive Origin (0).
2. **Exponentiation and Logarithms** are paired processes relative to the Multiplicative Origin (1).
3. **Square Roots** are symmetric instances where the reductive operator magnitude equals the iteration count.

This framework suggests that algorithms for higher-order operations can be optimized by analyzing the “distance” to the identity element. While gaps remain—particularly in defining continuous iterations for Tetration (Level 3)—this perspective offers a robust interface between pure mathematics and algorithmic design.

## A Existence and Uniqueness

We briefly address the conditions under which a unique termination depth exists. Let the destructive process be defined by a continuous function  $f(t)$  with  $f(0) = a$ . If the destruction function  $\mathcal{D}(y, b)$  ensures that  $f(t)$  is **Monotonic** (strictly decreasing) and **Bounded** (crosses the Origin), then by the Intermediate Value Theorem, there exists a unique  $t_0$  such that  $f(t_0) = \mathcal{O}$ .

## References

- [1] Knuth, D. E. (1997). *The Art of Computer Programming, Vol 2: Seminumerical Algorithms*. Addison-Wesley.
- [2] Hennessy, J. L., & Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann.
- [3] Goodstein, R. L. (1957). *Recursive Number Theory*. North-Holland.
- [4] Muller, J. M. (2006). *Elementary Functions: Algorithms and Implementation*. Birkhäuser.